INSTITUTO FEDERAL DO TOCANTINS - CAMPUS COLINAS DO TOCANTINS
TECNOLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DANILO COSTA MATIAS

DESENVOLVIMENTO DE UM SIMULADOR DE XADREZ COM INTEGRAÇÃO DO MOTOR STOCKFISH UTILIZANDO GODOT

COLINAS DO TOCANTINS - TO
2026

DANILO COSTA MATIAS

DESENVOLVIMENTO DE UM SIMULADOR DE XADREZ COM INTEGRAÇÃO DO MOTOR STOCKFISH UTILIZANDO GODOT

Trabalho de curso apresentado ao curso de Tecnologo em Análise e Desenvolvimento de Sistemas do Instituto Federal do Tocantins – Campus Colinas do Tocantins, como requisito parcial para a obtenção do título de bacharel em Tecnologo em Análise e Desenvolvimento de Sistemas, sob orientação do Prof. Me. Fernando Turibio de Moura.

COLINAS DO TOCANTINS - TO

2026

DANILO COSTA MATIAS

DESENVOLVIMENTO DE UM SIMULADOR DE XADREZ COM INTEGRAÇÃO DO MOTOR STOCKFISH UTILIZANDO GODOT

Trabalho de curso apresentado ao curso de Tecnologo em Análise e Desenvolvimento de Sistemas do Instituto Federal do Tocantins – Campus Colinas do Tocantins, como requisito parcial para a obtenção do título de bacharel em Tecnologo em Análise e Desenvolvimento de Sistemas, sob orientação do Prof. Me. Fernando Turibio de Moura.

Banca Examinadora

Me. Fernando Turibio de Moura
Instituto Federal do Tocantins Campus Colinas do Tocantins

TITULO. NOME MEMBRO 1
INTITUIÇÃO DO MEMBRO 1

TITULO. NOME MEMBRO 2
INSTITUIÇÃO DO MEMBRO 2

Aprovada em DATA DA DEFESA

Dedico este trabalho à minha esposa, pelo apoio incondicional, e a todos que acreditaram no meu potencial mesmo nos momentos mais desafiadores desta jornada.

AGRADECIMENTOS

Primeiramente, a Deus, pela força, resiliência e clareza mental concedidas ao longo de toda esta jornada, permitindo-me superar os momentos de cansaço e os desafios inerentes à pesquisa e ao desenvolvimento de software.

Ao Instituto Federal de Educação, Ciência e Tecnologia do Tocantins (IFTO) - Campus Colinas do Tocantins, por ser o alicerce da minha formação profissional. Aos excelentes professores do curso de Tecnologia em Análise e Desenvolvimento de Sistemas (TADS), que não apenas compartilharam seus conhecimentos teóricos e práticos, mas também me inspiraram a buscar sempre a excelência e a inovação tecnológica.

Um agradecimento muito especial ao meu orientador, Prof. Me. Fernando Turibio de Moura. Sua paciência, dedicação e vasto conhecimento foram fundamentais para transformar uma ideia ambiciosa em um projeto real. Obrigado pela confiança no meu potencial, pelas revisões minuciosas e por me guiar com maestria pelas melhores práticas de engenharia de software, especialmente nos momentos mais complexos de integração deste simulador.

À minha esposa e à minha família, pela compreensão inesgotável nas inúmeras noites, madrugadas e finais de semana dedicados à programação, à correção de bugs e à escrita deste documento. Sem o suporte emocional, o incentivo constante e a paciência de vocês com as minhas ausências, a conclusão desta etapa não seria possível. Vocês são a minha maior motivação.

À gigantesca comunidade de desenvolvedores de software livre, em especial aos criadores e mantenedores da Godot Engine, da linguagem Python e do motor de xadrez Stockfish. A filosofia de código aberto, a documentação acessível e o compartilhamento global de conhecimento foram a base técnica que viabilizou a materialização e o sucesso deste sistema.

Aos meus colegas de turma e amigos de curso, por todas as linhas de código revisadas em conjunto, pelos testes incansáveis do jogo em suas diversas versões, pelas partidas jogadas e pelas horas de estudo compartilhadas. Vocês tornaram os desafios desta graduação muito mais leves e a jornada muito mais inesquecível.

Por fim, a todos que, de forma direta ou indireta, contribuíram com palavras de incentivo, críticas construtivas e apoio ao longo destes anos de graduação, meu mais sincero muito obrigado.

Quero que os jogadores experimentem coisas que só são possíveis dentro dos videogames e que nunca foram feitas antes. Do contrário, não há sentido em criar o jogo.

Hideo Kojima

RESUMO

Este trabalho apresenta o desenvolvimento de um simulador de xadrez com integração ao motor de inteligência artificial Stockfish, utilizando a Godot Engine. O objetivo principal consistiu em construir uma aplicação funcional, customizável e com alto nível competitivo, solucionando o desafio técnico de integrar uma inteligência artificial complexa a um motor de jogos moderno. A metodologia adotada envolveu a criação da interface gráfica e da lógica de validação de regras, como o roque e o *en passant*, utilizando a linguagem nativa GDScript. Para o processamento das jogadas do oponente virtual, utilizou-se a linguagem Python como *middleware*, responsável por estabelecer a comunicação assíncrona entre o frontend na Godot e o motor Stockfish, operando através do protocolo *Universal Chess Interface* (UCI). Adicionalmente, foi implementado um sistema dinâmico de temas visuais baseado em dicionários e matrizes de dados, permitindo a personalização da interface gráfica em tempo real. Os resultados obtidos demonstram que a arquitetura proposta garante fluidez na jogabilidade sem comprometer o desempenho computacional durante o cálculo das melhores jogadas. Conclui-se que a integração entre Godot, Python e Stockfish forma uma base robusta, eficiente e altamente modular para o desenvolvimento de jogos de tabuleiro digitais de alta complexidade técnica.

Palavras-chave: Xadrez. Godot Engine. Python. Inteligência Artificial. Stockfish.

ABSTRACT

This work presents the development of a chess simulator integrated with the Stockfish artificial intelligence engine, using the Godot Engine. The main objective was to build a functional, customizable, and highly competitive application, solving the technical challenge of integrating a complex artificial intelligence into a modern game engine. The adopted methodology involved creating the graphical interface and the rule validation logic, such as castling and *en passant*, using the native GDScript language. To process the virtual opponent's moves, the Python language was used as *middleware*, responsible for establishing asynchronous communication between the Godot frontend and the Stockfish engine, operating through the *Universal Chess Interface* (UCI) protocol. Additionally, a dynamic visual theme system based on dictionaries and data arrays was implemented, allowing for real-time customization of the graphical interface. The results obtained demonstrate that the proposed architecture ensures fluid gameplay without compromising computational performance during the calculation of the best moves. It is concluded that the integration between Godot, Python, and Stockfish forms a robust, efficient, and highly modular foundation for the development of digital board games of high technical complexity.

Keywords: Chess. Godot Engine. Python. Artificial Intelligence. Stockfish.

LISTA DE ILUSTRAÇÕES

Figura 1 – Interface de partida contra o computador na plataforma Lichess. 19
Figura 2 – Diagrama de Implantação da arquitetura do Royal Legacy. 24
Figura 3 – Rascunho da interface do Menu Principal. 25
Figura 4 – Rascunho da interface de configurações de áudio, dificuldade e temas. . . . 26
Figura 5 – Rascunho da interface principal do tabuleiro e painéis laterais. 27
Figura 6 – Diagrama Entidade Relacionamento para persistência local. 28
Figura 7 – Diagrama de Casos de Uso do simulador. 29
Figura 8 – Diagrama de Classes detalhando a estrutura GDScript e middleware Python. 30
Figura 9 – Interface final do Menu Principal e seleção de temas. 33
Figura 10 – Interface final do tabuleiro de jogo com tema visual aplicado. 34

LISTA DE TABELAS

Tabela 1 – Desempenho da comunicação assíncrona e estabilidade gráfica. 35

LISTA DE ABREVIATURAS E SIGLAS

SIGLA	SIGNIFICADO DA SIGLA

SUMÁRIO

1 INTRODUÇÃO **13**
1.1 Objetivo Geral 13
1.2 Objetivos específicos 14
1.3 Justificativa 14
1.4 Contribuições realizadas 15
1.5 Organização do trabalho 15
2 REVISÃO BIBLIOGRÁFICA **16**
2.1 Fundamentação teórica 16
2.1.1 Xadrez Computacional e Protocolos (FEN e UCI) 16
2.1.2 Godot Engine e GDScript 17
2.1.3 Linguagem Python 17
2.1.4 Motor de Inteligência Artificial Stockfish 17
2.2 Trabalhos correlatos 18
2.2.1 Godot With Me: Chess (Repositório Open-Source) 18
2.2.2 Lichess (Plataforma Web Open-Source) 18
2.2.3 Lucas Chess 19
2.2.4 Chess.com 19
3 MATERIAIS E MÉTODOS **20**
3.1 Materiais 20
3.1.1 Recursos de *Hardware* 20
3.1.2 Recursos de *Software* 21
3.2 Métodos 21
3.2.1 Levantamento de Requisitos e Modelagem 21
3.2.2 Etapas de Desenvolvimento 22
4 PROJETO DO SISTEMA **23**
4.1 Arquitetura do projeto 23
4.2 Protótipos 24
4.2.1 Menu Principal e Opções 25
4.2.2 Tabuleiro de Jogo 26
4.3 Diagrama Entidade Relacionamento (DER) 27
4.4 Diagrama de Casos de Uso 28

4.5 Diagrama de Classes 29
4.6 Algoritmos propostos 30
5 RESULTADOS E DISCUSSÃO **32**
5.1 Apresentação das interfaces finais 32
5.1.1 Menu Principal e Configurações 32
5.1.2 Tabuleiro e Gameplay 33
5.2 Avaliação de desempenho e tempo de resposta 34
5.3 Discussão dos resultados 35
6 CONSIDERAÇÕES FINAIS **36**
6.1 Contribuições 36
6.2 Trabalhos Futuros 36
APÊNDICE A TITULO **38**

1 INTRODUÇÃO

O xadrez é um dos jogos de tabuleiro mais antigos e estudados do mundo, sendo frequentemente utilizado ao longo da história como um laboratório para testar a capacidade lógica e o poder de processamento dos computadores. Nos primórdios da ciência da computação, as partidas contra a máquina eram restritas a terminais de texto escuros ou interfaces extremamente rudimentares. No entanto, com a evolução da tecnologia e a mudança nas expectativas dos jogadores, os jogos digitais passaram a exigir não apenas uma matemática impecável, mas também uma apresentação visual imersiva e responsiva.

Hoje em dia, jogar xadrez contra o computador é algo comum, mas o desenvolvimento desse tipo de software envolve um desafio de arquitetura bastante específico: unir uma interface visual atraente, rica em animações e menus, com um "cérebro"lógico capaz de calcular milhões de probabilidades matemáticas em poucos segundos.

Motores de jogos modernos, como a Godot Engine, são ferramentas excelentes e acessíveis para criar telas interativas, gerenciar efeitos sonoros e construir menus complexos. Contudo, eles não foram construídos para processar algoritmos pesados de inteligência artificial de forma nativa. Por outro lado, ferramentas consagradas de Inteligência Artificial voltadas para o xadrez, como o Stockfish, são extremamente poderosas e precisas, mas funcionam apenas nos bastidores, através de linhas de comando, sem oferecer nenhuma tela, gráfico ou botão para o jogador humano.

Diante desse cenário, este trabalho apresenta o desenvolvimento do simulador digital *Royal Legacy*. O problema de pesquisa que norteia este projeto é: como integrar de forma eficiente um motor lógico de xadrez de alta complexidade (Stockfish) a uma interface gráfica moderna (Godot Engine), garantindo que o jogo seja visualmente personalizável e que os cálculos da máquina não congelem ou travem a experiência do usuário durante as partidas?

1.1 Objetivo Geral

Este trabalho tem como objetivo desenvolver um simulador de xadrez digital interativo e customizável na Godot Engine, integrado ao motor de inteligência artificial Stockfish através de uma comunicação em linguagem Python, proporcionando uma experiência de jogo fluida, modular e com diferentes níveis de dificuldade.

1.2 Objetivos específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Implementar a interface gráfica e a lógica completa das regras do xadrez (incluindo movimentação restrita, roque e *en passant*) utilizando a linguagem nativa GDScript;
- Desenvolver um *middleware* em Python para estabelecer a comunicação assíncrona entre o frontend do jogo e o binário do Stockfish, operando através do protocolo *Universal Chess Interface* (UCI);
- Estruturar um sistema de gerenciamento de cenas e menus de opções, garantindo uma experiência de usuário (UX) intuitiva para a configuração de volume, dificuldades e modos de jogo;
- Criar um sistema dinâmico de temas visuais baseado em dicionários de dados, permitindo a alteração das cores e dos conjuntos de peças em tempo real;
- Desenvolver um gerenciador de estados para controlar turnos e condições de fim de jogo (xeque-mate e empate), validando o fluxo correto das partidas nas modalidades Jogador contra Jogador (PvP) e Jogador contra Máquina (PvE).

1.3 Justificativa

A relevância deste trabalho reside na demonstração prática de uma arquitetura de software altamente modular. O desenvolvimento de um algoritmo de xadrez do zero, com alta performance, diretamente em uma *Game Engine* seria inviável para o escopo do projeto, além de computacionalmente ineficiente. Ao invés de reinventar ferramentas já consolidadas pela comunidade científica e de desenvolvedores, este projeto propõe uma ponte técnica elegante entre tecnologias de naturezas distintas (uma engine gráfica, uma linguagem de script utilitária e um executável externo).

Essa abordagem arquitetural é fortemente justificável pois resolve uma lacuna comum no desenvolvimento de software de entretenimento: a dificuldade de executar processos pesados em segundo plano sem impactar a fluidez visual do sistema. Muitos desenvolvedores iniciantes enfrentam travamentos de tela ao tentar calcular lances de IA na mesma via de processamento (*thread*) dos gráficos. Ao documentar a separação dessas responsabilidades, este projeto serve

como um modelo de estudo valioso para estudantes de Análise e Desenvolvimento de Sistemas que precisem integrar motores visuais modernos a inteligências artificiais complexas.

1.4 Contribuições realizadas

O desenvolvimento deste simulador trouxe contribuições práticas e metodológicas para a estruturação de projetos de jogos de tabuleiro, destacando-se:

- **Arquitetura de Comunicação Assíncrona:** A implementação de um script em Python que atua como ponte, traduzindo o estado do tabuleiro (formato FEN) e devolvendo a melhor jogada (formato UCI), provou que é viável acoplar programas externos pesados à Godot sem causar interrupções (*stutters*) na interface principal.
- **Modularidade de Interface:** A centralização da navegação entre os diversos menus e telas do jogo por meio de um único gerenciador global demonstrou a aplicação de boas práticas de design de software, facilitando a expansão do código e evitando o acoplamento excessivo entre as telas.
- **Escalabilidade Visual:** A criação de um sistema de customização que utiliza estruturas de dados flexíveis (como Dicionários) substituiu lógicas condicionais repetitivas, resultando em um código mais limpo e otimizado para futuras atualizações de conteúdo estético.

1.5 Organização do trabalho

Este capítulo expôs o contexto inicial, o problema de pesquisa, os motivos, os objetivos e as contribuições deste trabalho. O capítulo 2 apresenta os principais conceitos teóricos sobre o xadrez computacional, assim como as ferramentas, linguagens e *frameworks* utilizados para a solução do problema. O capítulo 3 apresenta a metodologia adotada e a arquitetura delineada para o desenvolvimento da aplicação. O capítulo 4 expõe o projeto de *software* construído, detalhando os scripts, a organização visual e o fluxo de dados entre o jogo e a Inteligência Artificial. O capítulo 5 apresenta os resultados práticos obtidos com o simulador em funcionamento, discutindo a eficiência da comunicação assíncrona. Por fim, o capítulo 6 compila as considerações finais sobre os desafios superados e sugere propostas para trabalhos futuros.

2 REVISÃO BIBLIOGRÁFICA

Este capítulo apresenta a base teórica e mercadológica que fundamenta o desenvolvimento do simulador *Royal Legacy*. Para compreender a arquitetura do projeto, é necessário explorar os conceitos tecnológicos aplicados na sua construção, bem como analisar softwares já existentes que resolvem problemas semelhantes.

A seção 2.1 dedica-se à fundamentação teórica, descrevendo os conceitos de xadrez computacional, os protocolos de comunicação padronizados e as tecnologias (motores e linguagens de programação) escolhidas para o desenvolvimento. Em seguida, a seção 2.2 apresenta uma análise de trabalhos correlatos e plataformas de xadrez amplamente utilizadas no mercado, destacando as diferenças e justificando as escolhas arquiteturais deste projeto em relação ao cenário atual.

2.1 Fundamentação teórica

A fundamentação teórica estabelece os conceitos e as ferramentas obtidos na literatura técnica que viabilizaram a construção do simulador. Segundo **??**), esta etapa é essencial para embasar cientificamente as decisões de projeto. A seguir, são detalhados os protocolos e *frameworks* centrais do trabalho.

2.1.1 Xadrez Computacional e Protocolos (FEN e UCI)

O desenvolvimento de interfaces para xadrez exige a padronização da forma como o tabuleiro e as jogadas são representados pelo computador. A notação FEN (*Forsyth-Edwards Notation*) é o padrão global para descrever as posições de um jogo de xadrez em uma única linha de texto. Ela mapeia quais peças estão em quais casas, de quem é o turno e as disponibilidades de roque (**??**).

Além da representação do tabuleiro, a comunicação com motores de Inteligência Artificial requer uma linguagem comum. O protocolo UCI (*Universal Chess Interface*) é um padrão aberto que permite que interfaces gráficas (GUI) conversem com motores de xadrez de forma independente. O motor recebe a posição atual via texto e devolve a melhor jogada calculada, garantindo a separação entre a lógica visual e a lógica matemática profunda.

2.1.2 Godot Engine e GDScript

A Godot Engine é um motor de jogo (*Game Engine*) de código aberto e multiplataforma, amplamente reconhecido pela sua arquitetura flexível baseada em nós (*nodes*) e cenas. Diferente de outras ferramentas do mercado, a Godot oferece um ambiente unificado para o desenvolvimento 2D e 3D, sendo altamente otimizada para a criação de interfaces de usuário (UI) complexas (**??**).

Para a manipulação da lógica interna do jogo, utilizou-se o GDScript, a linguagem de programação nativa da Godot. O GDScript possui uma sintaxe inspirada em Python, sendo dinamicamente tipada e profundamente integrada ao motor, o que facilita o acesso rápido aos elementos visuais, instanciamento de cenas e manipulação de estruturas de dados, como os Dicionários utilizados para a troca de temas no *Royal Legacy*.

2.1.3 Linguagem Python

Python é uma linguagem de programação de alto nível, interpretada e de propósito geral, conhecida por sua legibilidade e vasta biblioteca padrão (**??**). Neste projeto, Python foi escolhido para atuar como um *middleware* (ponte) devido à sua robustez na manipulação de processos do sistema operacional e fluxos de entrada e saída de dados. Através de bibliotecas específicas, o script Python executa o binário da Inteligência Artificial em segundo plano, recebe os comandos da interface na Godot, formata os dados e os repassa via protocolo UCI, resolvendo os problemas de travamento da interface gráfica.

2.1.4 Motor de Inteligência Artificial Stockfish

O Stockfish é atualmente um dos motores de xadrez de código aberto mais fortes do mundo. Em vez de possuir uma interface visual própria, ele é um programa executável operado inteiramente por linha de comando. Nas suas versões mais recentes, o Stockfish combina algoritmos tradicionais de busca em profundidade, como o *Alpha-Beta Pruning*, com avaliações baseadas em redes neurais eficientes (NNUE), permitindo-lhe avaliar milhões de posições por segundo com precisão sobre-humana (**??**).

2.2 Trabalhos correlatos

Esta seção apresenta os principais softwares, plataformas e repositórios relacionados ao tema deste estudo. A análise destes sistemas permite não apenas identificar os padrões da indústria, mas também reconhecer as bases comunitárias que apoiaram o desenvolvimento arquitetural do simulador *Royal Legacy*.

2.2.1 Godot With Me: Chess (Repositório Open-Source)

O projeto *Godot With Me: Chess* é um repositório de código aberto hospedado no GitHub que serve como um guia prático para a implementação das regras básicas de xadrez utilizando a Godot Engine. Ele aborda a movimentação em *grid* (grade) e o instanciamento das peças 2D no tabuleiro (**????**).

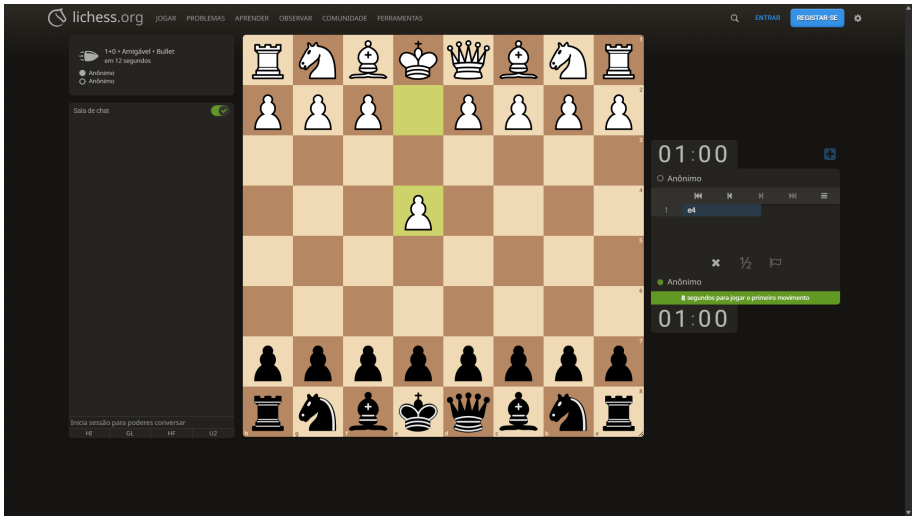
Este repositório serviu como principal referência inicial para o desenvolvimento do *Royal Legacy*, auxiliando no entendimento da manipulação de matrizes em GDScript e na estruturação visual das casas lógicas. No entanto, o presente trabalho diferencia-se e expande significativamente o escopo desse repositório base ao implementar três grandes melhorias estruturais: a integração de uma Inteligência Artificial externa (Stockfish) operando em segundo plano via Python; a criação de um gerenciador de cenas estruturado para menus complexos; e um sistema dinâmico de troca de temas visuais via Dicionários de dados.

2.2.2 Lichess (Plataforma Web Open-Source)

O Lichess é um dos maiores servidores de xadrez da Internet, sendo totalmente gratuito e de código aberto. A plataforma permite partidas *multiplayer* e contra a máquina, utilizando o próprio motor Stockfish nos seus servidores (*backend*) ou através da compilação em WebAssembly no navegador do usuário (*frontend*).

A figura 1 apresenta o tabuleiro padrão da plataforma. Embora seja uma referência mundial, o Lichess possui uma arquitetura focada em tecnologias web (cliente-servidor). O presente trabalho difere por ser uma aplicação *desktop* nativa focada em renderização através de uma *Game Engine*, dispensando a necessidade de navegadores para a integração com a Inteligência Artificial.

Figura 1 – Interface de partida contra o computador na plataforma Lichess.



2.2.3 Lucas Chess

O *Lucas Chess* é um software *desktop* voltado para o treinamento e ensino de xadrez. O seu grande diferencial é a inclusão de dezenas de motores de IA diferentes, configurados para simular adversários de diversas faixas de força (rating ELO), de iniciantes a Grandes Mestres.

Assim como o projeto *Royal Legacy*, o Lucas Chess utiliza a integração local com motores via protocolo UCI. No entanto, a sua interface gráfica é construída com ferramentas tradicionais de janelas do sistema operacional, focando mais na utilidade estatística do que na experiência audiovisual ou fluidez de animações. O uso da Godot Engine neste TCC propõe justamente uma evolução na apresentação visual e interativa desse tipo de simulador local.

2.2.4 Chess.com

O Chess.com é a plataforma comercial líder de mercado em xadrez digital. A aplicação oferece uma vasta gama de personalização de temas, avatares (bots) com personalidades distintas e análises profundas de partidas. Por ser um software proprietário, os motores lógicos utilizados por trás dos seus "bots"são fechados em servidores remotos de alto desempenho. O desenvolvimento do *Royal Legacy* busca espelhar a experiência de customização visual e a seleção de dificuldades de IA encontrada no Chess.com, porém arquitetando tudo de forma local, *offline* e utilizando apenas ferramentas *open-source*.

3 MATERIAIS E MÉTODOS

Este capítulo descreve a metodologia aplicada no desenvolvimento do simulador *Royal Legacy*. Serão apresentados os materiais tecnológicos, abrangendo *hardware* e *software*, e os métodos adotados para a modelagem, levantamento de requisitos e execução das etapas do projeto. Ressalta-se que a fundamentação teórica sobre o funcionamento destas tecnologias já foi abordada na seção 2.1.

3.1 Materiais

Os materiais empregados nesta pesquisa dividem-se em recursos físicos (*hardware*), utilizados para o desenvolvimento e medição de desempenho, e lógicos (*software*), que compõem o ecossistema de programação e integração do sistema.

3.1.1 Recursos de *Hardware*

O desenvolvimento da aplicação, bem como os testes de estresse envolvendo o cálculo de jogadas pela Inteligência Artificial, foram conduzidos em um computador (nomeado na rede local como SERVIDOR) com as seguintes especificações:

- **Processador:** AMD Ryzen 5 4600G com Radeon Graphics (3.70 GHz), arquitetura baseada em x64, contando com 6 núcleos e 12 *threads*. Essa configuração foi essencial para suportar a execução da *Game Engine* na via principal (*main thread*) em paralelo com o motor de xadrez em segundo plano, sem causar gargalos de processamento;
- **Memória RAM:** 16,0 GB de memória instalada (sendo 15,4 GB utilizáveis), garantindo a alocação necessária para a manipulação de matrizes do tabuleiro e o armazenamento da árvore de busca profunda do Stockfish;
- **Sistema Operacional e Arquitetura:** Sistema operacional de 64 bits, adequado para compilar o executável da Godot Engine e lidar com o alto volume de instruções da Inteligência Artificial;
- **Armazenamento:** Unidade de armazenamento local em disco rígido/SSD, utilizada para reduzir o tempo de carregamento dos *assets* visuais (temas e peças) e acelerar as chamadas de leitura do *script* Python.

3.1.2 Recursos de *Software*

As tecnologias e ferramentas de *software* foram aplicadas de forma modularizada, separando a interface gráfica da lógica computacional:

- **Godot Engine e GDScript:** O motor gráfico foi empregado como o Ambiente de Desenvolvimento Integrado (IDE) principal para a interface. O GDScript foi a linguagem utilizada para programar as regras de movimentação, o gerenciamento de cenas (`estruturador.tscn`) e o controle de áudio (`tela_sons.gd`);
- **Python:** Utilizado exclusivamente para o desenvolvimento do *middleware* (`chess_bridge.py`). O Python operou a biblioteca `subprocess` para abrir fluxos de leitura e escrita com a Inteligência Artificial;
- **Stockfish:** Empregado como o executável (*backend*) que recebe os estados do tabuleiro e devolve os cálculos matemáticos das melhores jogadas durante as partidas contra a máquina;
- **Visual Studio Code:** Utilizado como IDE auxiliar para a escrita, formatação e depuração do código Python, facilitando a visualização dos retornos do terminal;
- **Protocolos FEN e UCI:** O padrão FEN foi utilizado como estrutura de dados para trafegar o posicionamento das peças entre a Godot e o Python. O protocolo UCI funcionou como a Interface de Programação de Aplicações (API) padrão para enviar comandos de nível de dificuldade ao Stockfish.

3.2 Métodos

O projeto foi conduzido utilizando a metodologia de desenvolvimento iterativo e incremental. Essa escolha metodológica permitiu que o simulador de xadrez fosse construído em pequenos ciclos funcionais, garantindo que a base do jogo estivesse sólida antes da introdução de sistemas mais complexos, como a Inteligência Artificial.

3.2.1 Levantamento de Requisitos e Modelagem

Como o projeto trata-se da simulação de um jogo de tabuleiro clássico, o levantamento de requisitos não ocorreu por meio de entrevistas tradicionais com usuários, mas sim através

de pesquisa documental e análise de sistemas correlatos. Os requisitos de regras lógicas foram extraídos do manual oficial da Federação Internacional de Xadrez (FIDE). Já os requisitos arquiteturais basearam-se na documentação do protocolo UCI para a troca de mensagens de texto.

Para a modelagem do sistema, optou-se pela utilização de Diagramas de Fluxo de Dados (DFD) e fluxogramas lógicos, que mapearam o caminho da informação desde o clique do usuário na interface da Godot, passando pela tradução no *script* Python, até o processamento no binário do Stockfish e o seu retorno para a tela.

3.2.2 Etapas de Desenvolvimento

O desenvolvimento do *Royal Legacy* foi dividido em quatro iterações principais, executadas sequencialmente:

1. **Iteração 1 - Estruturação Visual:** Criação do tabuleiro em grade (*grid*), instanciamento das peças 2D e implementação da lógica básica de seleção e movimentação com o *mouse*;

2. **Iteração 2 - Lógica de Regras:** Codificação das validações geométricas no GDScript (movimentos permitidos por peça) e implementação de regras especiais (*en passant*, roque e promoção);

3. **Iteração 3 - Integração com a IA:** Construção do *script* ponte em Python, formatação do tabuleiro em FEN e estabelecimento da comunicação assíncrona com o Stockfish sem travar a interface visual;

4. **Iteração 4 - Polimento e Gerenciamento de Estado:** Criação do `estruturador.tscn` para transitar entre os menus, aplicação do Dicionário de dados para a troca dinâmica de temas visuais e ajustes finais no controlador de áudio.

4 PROJETO DO SISTEMA

Este capítulo apresenta os principais elementos de projeto do simulador *Royal Legacy*. São detalhadas a arquitetura do sistema, os protótipos das interfaces, a modelagem de dados para persistência, os diagramas estruturais e comportamentais baseados na Unified Modeling Language (UML), bem como a representação lógica dos algoritmos centrais de validação e comunicação.

As decisões de projeto aqui expostas visam atender aos requisitos de modularidade, customização visual e execução assíncrona da Inteligência Artificial, conforme justificado na seção 1.3.

4.1 Arquitetura do projeto

A arquitetura do *Royal Legacy* foi projetada seguindo o princípio da separação de responsabilidades, dividindo o sistema em camadas distintas que se comunicam através de protocolos padronizados. Essa abordagem é fundamental para garantir que o processamento pesado da IA não impacte a fluidez da interface gráfica (**??**).

A Figura 2 apresenta o diagrama de implantação que ilustra como as partes do sistema foram divididas e como elas se comunicam.

Figura 2 – Diagrama de Implantação da arquitetura do Royal Legacy.

./capitulos/projeto/arquitetura_implantaÃğÃčo.png

O fluxo de comunicação segue o modelo Cliente-Servidor local:

1. **Camada de Apresentação (Godot Engine):** Atua como o cliente (Frontend). Gerencia a interface, captura entradas do usuário e envia o estado do jogo (formato FEN) para o intermediário.

2. **Camada de Comunicação (*Middleware* Python):** Um *script* que roda em segundo plano, agindo como ponte. Ele orquestra a execução da IA e traduz os comandos.

3. **Camada Lógica (Stockfish Binary):** O motor de xadrez (Backend). Recebe configurações via protocolo UCI e devolve os cálculos matemáticos.

4.2 Protótipos

Nesta seção são apresentados os protótipos de baixa fidelidade das principais interfaces do *Royal Legacy*. Estes rascunhos serviram como base para o desenvolvimento visual, focando

na disposição dos elementos e na Experiência do Usuário (UX), não representando as telas finais com texturas e cores aplicadas.

4.2.1 Menu Principal e Opções

O protótipo do Menu Principal (Figura 3) foi desenhado para ser minimalista, oferecendo acesso rápido aos modos de jogo e configurações. O protótipo da tela de Opções (Figura 4) detalha os rascunhos dos seletores para volume, nível de dificuldade da IA (Fácil, Médio, Difícil) e a galeria de temas visuais.

Figura 3 – Rascunho da interface do Menu Principal.

./capitulos/projeto/prototipo_menu.png

Figura 4 – Rascunho da interface de configurações de áudio, dificuldade e temas.

./capitulos/projeto/prototipo_opcoes.png

4.2.2 Tabuleiro de Jogo

O protótipo da tela de jogo (Figura 5) apresenta a grade 8×8 centralizada, ladeada por rascunhos dos painéis de peças capturadas, cronômetro (opcional) e o botão para retornar ao menu.

Figura 5 – Rascunho da interface principal do tabuleiro e painéis laterais.

./capitulos/projeto/prototipo_jogo.png

4.3 Diagrama Entidade Relacionamento (DER)

Dado que o *Royal Legacy* é um simulador local focado na jogabilidade e não na gestão complexa de usuários ou grandes volumes de dados relacionais, optou-se por não utilizar um Banco de Dados Relacional tradicional (como PostgreSQL ou MySQL). Em vez disso, a persistência de dados é realizada através de arquivos de configuração locais (formato JSON ou dicionários serializados na Godot), o que simplifica a arquitetura e reduz o consumo de recursos da máquina (**??**).

No entanto, para representar como as estruturas de dados necessárias para o funcionamento do sistema foram modeladas de forma lógica e normalizada, apresenta-se na Figura 6 o Diagrama Entidade Relacionamento (DER) que rege a persistência das configurações e estados de partidas.

Figura 6 – Diagrama Entidade Relacionamento para persistência local.

./capitulos/projeto/der_jogo.png

O DER segue as normas formais de modelagem:

- **Configuracao_Local:** Armazena as preferências do usuário (Volume, Dificuldade, ID_Tema_Ativo).
- **Tema_Visual:** Estrutura os caminhos dos arquivos (*sprites*) e códigos de cores para a customização dinâmica da interface.
- **Historico_Partida (Opcional):** Permite salvar o estado atual do tabuleiro em formato FEN para retomada posterior.

Todos os campos visíveis nos protótipos da seção 4.2 (como seletores de volume e temas) possuem representação correspondente neste modelo de dados.

4.4 Diagrama de Casos de Uso

O Diagrama de Casos de Uso (Figura 7) descreve as funcionalidades do sistema sob a perspectiva dos atores envolvidos. No contexto do *Royal Legacy*, o ator principal é o **Jogador

Humano**, enquanto o **Motor Stockfish** atua como um ator secundário invocado pelo sistema nos modos PvE.

Figura 7 – Diagrama de Casos de Uso do simulador.

./capitulos/projeto/diagrama_uso.png

Os principais casos de uso incluem:

4.5 Diagrama de Classes

A organização estrutural do código na Godot Engine, embora baseada em Cenas e Nós, pode ser representada formalmente através de um Diagrama de Classes UML. A Figura 8 apresenta as principais classes de domínio do *Royal Legacy* e suas relações.

Figura 8 – Diagrama de Classes detalhando a estrutura GDScript e middleware Python.

./capitulos/projeto/diagrama_classes.png

Destacam-se as seguintes classes e relações:

- **StructuralManager:** Atua como o nó raiz (`estruturador.tscn`), gerenciando os estados de troca de cenas (Menus e Tabuleiro).
- **ChessBoard:** Classe central que detém a matriz do tabuleiro, o histórico de lances e a lógica de validação de regras.

4.6 Algoritmos propostos

Em conformidade com as diretrizes deste trabalho, esta seção não apresenta códigos-fonte brutos (GDScript ou Python), mas sim a representação lógica dos algoritmos que constituem contribuições claras para a solução do problema de pesquisa.

O Algoritmo 1 descreve a lógica implementada na Godot para solicitar a jogada da Inteligência Artificial sem travar a interface visual, utilizando a execução de subprocessos externos.

Algoritmo 1: Solicitação Assíncrona de Jogada da IA (Godot Side).

```
1  início
2      estado_fen
        ←obter_estado_atual_tabuleiro() dificuldade←obter_configuracao_dificuldade()
3      // Preparar o Middleware Python como processo externo
4      comando ←"python chess_bridge.py –fen "+ estado_fen + --diff "+ dificuldade
5      // Executar de forma assíncrona (thread separada)
6      se OS.execute_assincrono(comando, callback_funcao_retorno) então
7          bloquear_entrada_usuario_humano();
8          exibir_icone_processando();
9      fim
10 fim
```

O Algoritmo 2 representa a lógica central do *middleware* Python, que gerencia a comunicação via *pipes* de terminal com o binário do Stockfish utilizando o protocolo UCI.

Algoritmo 2: Middleware Python de Comunicação com Stockfish (UCI Protocol).

```
   Entrada: arg_fen (String), arg_diff (String)
   Saída: Melhor jogada (String formato "e2e4")
1  início
2      // Abrir processo binário do Stockfish com pipes de leitura/escrita
3      stockfish ←subprocess.Popen("./stockfish", stdin=PIPE, stdout=PIPE)(??)
4      // Configurar dificuldade e posição via UCI
5      enviar_comando_uci(stockfish, "setoptionnameSkillLevelvalue" + arg_diff) ;
6      enviar_comando_uci(stockfish, "positionfen" + arg_fen) ;
7      // Solicitar cálculo restringindo tempo ou profundidade
8      enviar_comando_uci(stockfish, "gomovetime2000");
9      // Ler saída do Stockfish aguardando a linha "bestmove"
10     linha ←ler_saida_pipe(stockfish) "bestmove" ∉ linha
11     linha ←ler_saida_pipe(stockfish)
12     // Extrair e retornar apenas as coordenadas da jogada
13     jogada_final
        ←extrair_coordenadas(linha) fechar_processo(stockfish) retorna jogada_final fim
14
```

5 RESULTADOS E DISCUSSÃO

Este capítulo destina-se a apresentar e debater os resultados obtidos após a implementação do simulador *Royal Legacy*. Serão expostas as interfaces gráficas finais, traçando um comparativo com os protótipos idealizados na fase de projeto, além da apresentação das métricas de desempenho que validam a eficiência da arquitetura de comunicação assíncrona proposta neste trabalho.

Conforme a metodologia definida para projetos que envolvem algoritmos e tempos de processamento, a avaliação principal centrou-se na medição do tempo de resposta do sistema e na estabilidade da taxa de quadros por segundo (FPS) durante a execução de tarefas computacionalmente pesadas.

5.1 Apresentação das interfaces finais

A interface gráfica de um jogo é o principal ponto de contato com o usuário. Durante o desenvolvimento na Godot Engine, os protótipos de baixa fidelidade (apresentados na seção 4.2) foram refinados com a aplicação de texturas, paletas de cores harmônicas e tipografia adequada, resultando nas telas finais do sistema.

5.1.1 Menu Principal e Configurações

A Figura 9 apresenta a tela final do Menu Principal e de Opções. Em comparação com o rascunho inicial, a versão final implementa o sistema de temas baseado em Dicionários de forma visual e interativa. O usuário pode visualizar a prévia das cores do tabuleiro em tempo real ao selecionar um novo tema, demonstrando a eficácia do carregamento dinâmico de *sprites* via GDScript.

Figura 9 – Interface final do Menu Principal e seleção de temas.

./capitulos/resultados_discussao/tela_menu_final.png

5.1.2 Tabuleiro e Gameplay

A Figura 10 exibe o tabuleiro de xadrez durante uma partida na modalidade Jogador contra Máquina (PvE). A interface final manteve a coesão com o protótipo estrutural, mas adicionou indicadores visuais essenciais para a utilidade do sistema, como o realce da casa selecionada e a exibição das últimas peças movimentadas, melhorando significativamente a clareza (*feedback* visual) para o jogador humano.

Figura 10 – Interface final do tabuleiro de jogo com tema visual aplicado.

./capitulos/resultados_discussao/tela_jogo_final.png

5.2 Avaliação de desempenho e tempo de resposta

O problema central desta pesquisa consistia em integrar o motor de Inteligência Artificial (Stockfish) à interface gráfica sem causar travamentos (*stutters*) na via principal de renderização. Para validar esta solução, o sistema foi submetido a testes de estresse no ambiente de *hardware* descrito na seção **??**.

Os testes consistiram em medir o tempo que o *middleware* Python levou para retornar a jogada calculada pelo Stockfish em três níveis de dificuldade distintos (baseados na profundidade da árvore de busca do algoritmo da IA), enquanto a taxa de atualização da tela da Godot (FPS) era monitorada. Os resultados obtidos estão consolidados na Tabela 1.

Tabela 1 – Desempenho da comunicação assíncrona e estabilidade gráfica.

Nível de Dificuldade	Profundidade de Busca	Tempo de Resposta Médio	Estabilidade da
Fácil	5 lances	0,15 segundos	Constan
Médio	10 lances	1,20 segundos	Constan
Difícil	15 lances	3,45 segundos	Constan

Elaborado pelo autor (2026).

5.3 Discussão dos resultados

A análise da Tabela 1 confirma que a arquitetura adotada resolveu o problema de pesquisa com êxito. Mesmo na dificuldade "Difícil", onde o Stockfish precisou processar milhões de posições, levando cerca de 3,45 segundos para retornar uma string via protocolo UCI, a interface desenvolvida na Godot Engine manteve-se perfeitamente estável, registrando 60 quadros por segundo.

Se a comunicação não tivesse sido isolada por meio do *script* assíncrono em Python (conforme planejado nos métodos deste trabalho), a tela do jogo teria congelado completamente durante esses 3 segundos, impossibilitando até mesmo a execução das animações e dos efeitos de áudio. O *middleware* provou ser não apenas uma ponte funcional de tradução do formato FEN, mas também um isolante de processos (*sandbox*) altamente eficaz.

Por fim, no aspecto da usabilidade, a modularidade do nó `estruturador.tscn` garantiu que a transição entre o Menu e o Jogo ocorresse de forma instantânea, validando as boas práticas de design de *software* adotadas ao não carregar todas as cenas simultaneamente na memória RAM.

6 CONSIDERAÇÕES FINAIS

Este capítulo apresenta as considerações finais do trabalho, sumarizando os resultados obtidos com o desenvolvimento do simulador *Royal Legacy*. O projeto alcançou o seu objetivo principal de criar uma experiência de xadrez digital fluida, integrando um motor lógico de alto desempenho sem comprometer a estabilidade da interface gráfica. O capítulo divide-se nas seções 6.1, que detalha os avanços técnicos alcançados, e 6.2, que sugere caminhos para a continuidade desta pesquisa.

6.1 Contribuições

O desenvolvimento deste projeto resultou em contribuições técnicas e arquiteturais significativas para a criação de jogos de tabuleiro integrados a motores externos. Destacam-se pontualmente:

* **Arquitetura Assíncrona de Comunicação:** A criação de um *middleware* em Python demonstrou ser uma solução altamente eficaz para o problema de travamento de interface (*stuttering*). A execução isolada do binário do Stockfish permitiu que a Godot Engine mantivesse uma taxa de renderização constante de 60 FPS durante o cálculo de jogadas profundas.
* **Motor Lógico Independente em GDScript:** A transcrição de todas as regras do xadrez (incluindo lances especiais como *en passant*, roque e promoção) para a linguagem nativa da Godot, utilizando matrizes bidimensionais, resultou em um código base modular e de fácil manutenção.
* **Sistema de Customização Dinâmica:** A implementação de um gerenciador de temas baseado em Dicionários de dados otimizou a troca de recursos visuais (*sprites* e paletas de cores) em tempo de execução, dispensando o uso de estruturas condicionais extensas e reduzindo o consumo desnecessário de memória.

6.2 Trabalhos Futuros

Apesar do pleno funcionamento da arquitetura proposta e do atingimento dos objetivos iniciais, o escopo do *Royal Legacy* permite diversas expansões. Para que

futuros pesquisadores ou desenvolvedores continuem contribuindo com o tema, sugerem-se as seguintes melhorias a médio e longo prazo:

* **Implementação de *Multiplayer* Online:** Expandir o sistema para suportar partidas remotas entre jogadores (PvP via rede), utilizando a API de rede de alto nível da Godot ou WebSockets para a troca eficiente de *strings* FEN através de um servidor central.
* **Portabilidade para Dispositivos Móveis:** Adaptar a interface para telas sensíveis ao toque (UX/UI *mobile*) e reestruturar a comunicação assíncrona para compilar o código-fonte do Stockfish nativamente como um módulo em C++ (GDExtension), permitindo a execução em sistemas Android e iOS sem a necessidade do interpretador Python.
* **Integração com Múltiplos Motores (IA):** Adicionar suporte a outras Inteligências Artificiais de xadrez de código aberto compatíveis com o protocolo UCI, como a Leela Chess Zero (baseada em redes neurais), permitindo que o usuário enfrente oponentes virtuais com diferentes "personalidades"e estilos de jogo.
* **Análise de Partidas em Tempo Real:** Desenvolver um módulo gráfico que traduza a avaliação numérica contínua do Stockfish em uma barra de vantagens visual durante o jogo, auxiliando o usuário no aprendizado, estudo de posições e detecção de erros graves (*blunders*).

APÊNDICE A – TITULO